Sensitivity Analysis for Irregular Assessment Times with General Outcome Types

An example using simulated ARC trial data

Contents

1 Overview **2**
- 1.1 Brief review of the method . . . 2

2 Setup **2**
- 2.1 Working directory . . . 2
- 2.2 Packages and functions . . . 3
- 2.3 Running time . . . 3

3 Data **4**

4 Estimation of the mean outcome **5**
- 4.1 Point estimates and influence-function-based variance . . . 5
- 4.2 Parametric bootstrap confidence intervals . . . 6

5 Visualization **7**
- 5.1 Estimated mean curves . . . 7
- 5.2 Confidence intervals at the target times . . . 9
- 5.3 Contour plots of the treatment effect . . . 11
  - 5.3.1 Estimation on a fine grid of α . . . 11
  - 5.3.2 Treatment effect estimates and confidence intervals . . . 13
  - 5.3.3 Contour plot of the point estimates . . . 14
  - 5.3.4 Contour plot of the confidence intervals . . . 15

6 Selection of the sensitivity parameter **18**
- 6.1 Point estimates and bias-corrected point estimates . . . 18
  - 6.1.1 Algorithm 1: original point estimates . . . 18
  - 6.1.2 Algorithm 2: multiplier bootstrap bias correction . . . 19
- 6.2 Confidence set for (α_a^l, α_a^u) . . . 20
  - 6.2.1 Algorithm 3: double bootstrap . . . 20

7 Other outcome types and options **24**
- 7.1 Binary outcome (AC indicator) . . . 24
- 7.2 Summary of main options . . . 25

8 Session information **25**

1 Overview

This vignette shows how to use our code to carry out the sensitivity analysis proposed in the paper, for randomized trials in which the outcome is assessed at **irregular times** and assessment may depend on the outcome itself. The code supports general outcome types through a link function (identity, log, or logit).

We use a simulated data set generated from the Asthma Research for the Community (ARC) trial, following the simulation design in the paper. The vignette covers:

1. **Estimation of the mean outcome** $\mu^a(t)$ in each arm for fixed values of the sensitivity parameter α_a, with influence-function-based and parametric-bootstrap confidence intervals.
2. **Visualization**: the estimated mean curves, confidence intervals at the target times (3, 6, 9, and 12 months), and contour plots of the treatment effect $\delta(t) = \mu^1(t) - \mu^0(t)$ over a grid of (α_0, α_1).
3. **Selection of the sensitivity parameter**: point estimates (Algorithm 1), multiplier bootstrap bias-corrected estimates (Algorithm 2), and double bootstrap confidence sets (Algorithm 3) for the range (α_a^l, α_a^u).

1.1 Brief review of the method

For each arm a, the sensitivity parameter α_a enters through the exponential tilting assumption

$$dF\{y(t) \mid \Delta N(t) = 0, \overline{O}(t)\} \propto \exp(\alpha y)\, dF\{y(t) \mid \Delta N(t) = 1, \overline{O}(t)\},$$

so that a negative (positive) α_a means that unobserved outcomes at time t tend to be lower (higher) than observed outcomes among similar participants. Setting $\alpha_a = 0$ gives the explainable assessment assumption.

The mean outcome is modeled as $\mu(t) = s\{B(t)'\beta\}$ for $t \in [t_1, t_2]$, where $B(t)$ is a cubic spline basis and $s = g^{-1}$ is a known inverse link function. Here we use $[t_1, t_2] = [60, 460]$ days and a spline with one interior knot at $t = 260$. The coefficient β is estimated by an augmented inverse intensity-weighted (AIIW) estimator based on the influence function in the paper, with one of two weight functions, $W_1(t; \beta)$ or $W_2(t; \beta)$.

Two observed-data models are fitted within each arm:

- the assessment intensity $\lambda\{t \mid \overline{O}(t)\}$, using an Andersen–Gill model stratified by assessment number; and
- the conditional distribution of observed outcomes, using a single-index model with predictors `Prev_outcome`, `time`, and `Lag_time`.

To choose plausible values of α_a, clinical experts give a plausible range (μ_a^l, μ_a^u) for the mean outcome. Based on the monotonicity of $\mu^a(t; \alpha_a)$ in α_a, the set of values for which $\mu_a^l < \mu^a(t; \alpha_a) < \mu_a^u$ for all $t \in [t_1, t_2]$ is an interval (α_a^l, α_a^u), which we estimate from the data.

2 Setup

2.1 Working directory

The working directory is set to the folder that contains this R Markdown file, so the function files (1. function_inference.R and 1. function_selection.R) and the simulated data sets should be placed in

the same folder. When knitting, setwd() only affects the chunk it is called in, so we also set root.dir, which makes all later chunks run in this folder.

```
if (isTRUE(getOption("knitr.in.progress"))) {
  # when knitting
  path <- dirname(knitr::current_input(dir = TRUE))
} else if (requireNamespace("rstudioapi", quietly = TRUE) &&
           rstudioapi::isAvailable()) {
  # when running chunks interactively in RStudio
  path <- dirname(rstudioapi::getSourceEditorContext()$path)
} else {
  # otherwise, use the current working directory
  path <- getwd()
}

setwd(path)
knitr::opts_knit$set(root.dir = path)
getwd()
```

```
## [1] "/Users/jane.gao/Downloads/data_analysis_norm"
```

2.2 Packages and functions

All functions for estimation and inference are stored in `1. function_inference.R`. All functions for sensitivity parameter selection are stored in `1. function_selection.R`. The single-index model is fitted with functions from the `SensIAT` package.

```
library(dplyr)
library(survival)
library(MAVE)
library(dfoptim)
library(orthogonalsplinebasis)
library(BB)
library(statmod)   # Gauss-Legendre quadrature
library(parallel)
library(ggplot2)
library(metR)      # contour plots
library(isoband)
library(SensIAT)   # single-index model

# functions for estimation and inference
source("1. function_inference.R")
# functions for sensitivity parameter selection
source("1. function_selection.R")
```

2.3 Running time

Several steps (parametric bootstrap, multiplier bootstrap, and especially the double bootstrap) are computationally intensive and are designed to run in parallel with `parallel::mclapply()`. The

number of cores is set by `n_cores` below. Note that `mclapply()` does not support more than one core on Windows.

To avoid re-running these steps every time the vignette is knitted, we use a small helper that saves each time-consuming result to the `cache` folder and loads it on later runs. Delete the corresponding `.rds` file to re-run a step. The helper also hides the progress messages printed by the estimation functions, so that they do not appear in the PDF.

```
n_cores   <- 96   # e.g., 64 or 96 on a cluster; use 1 on Windows
cache_dir <- "cache"
dir.create(cache_dir, showWarnings = FALSE, recursive = TRUE)

run_or_load <- function(name, expr) {
  file <- file.path(cache_dir, paste0(name, ".rds"))
  if (file.exists(file)) return(readRDS(file))
  # run only if no saved result exists; progress messages are hidden
  invisible(utils::capture.output(result <- expr))
  saveRDS(result, file)
  result
}
```

3 Data

The simulated data set is based on the ARC trial and is generated using the simulation method described in the paper. Each arm is stored as a separate data set:

- `ARC_simu_data_200_neg_trt` and `ARC_simu_data_200_neg_crl`: the original ACQ score for the intervention (PT+HV) arm and the control (PT) arm. The ACQ score ranges from 0 to 6, and either the log link or the identity link can be applied.
- `ARC_simu_data_200_logistic_trt` and `ARC_simu_data_200_logistic_crl`: the binary asthma control (AC) indicator, which should be analyzed with the logit link.

```
data_ARC_trt_continuous <- readRDS(file = "ARC_simu_data_200_neg_trt")
data_ARC_crl_continuous <- readRDS(file = "ARC_simu_data_200_neg_crl")

# binary AC indicator (logit link)
# data_ARC_trt_binary <- readRDS(file = "ARC_simu_data_200_logistic_trt")
# data_ARC_crl_binary <- readRDS(file = "ARC_simu_data_200_logistic_crl")

head(data_ARC_trt_continuous)
```

```
##     id time event Prev_outcome prev_time Lag_time visit  outcome
## 129  1  830     0     4.666667         0      830     1       NA
## 2    2  192     1     2.166667         0      192     1 2.500000
## 210  2  578     1     2.500000       192      386     2 1.666667
## 211  2  670     1     1.666667       578       92     3 2.000000
## 4    2  830     0     2.000000       670      160     4       NA
## 109  3  130     1     2.666667         0      130     1 1.333333
```

The data are in long format, with one row per assessment and one additional row at the end of follow-up (day 830) for each participant, where `event = 0` and `outcome = NA`. The variables are the participant identifier `id`, the assessment time `time` (days since randomization), the assessment indicator `event`, the outcome at the previous assessment `Prev_outcome`, the time of the previous assessment `prev_time`, the time since the previous assessment `Lag_time`, the assessment number `visit`, and the outcome `outcome`.

4 Estimation of the mean outcome

Throughout this section, we use the **log link** for the ACQ score and the first weight function $W_1(t;\beta)$ as an example. For other link functions or weight functions, change the `link_type` and `IF_type` arguments in each function.

Based on the selected range of the sensitivity parameter (see the last part of this vignette), we consider $\alpha \in \{-1.2, -0.8, -0.4, 0, 0.4, 0.8\}$ for the ACQ score.

```
Alpha_seq <- c(-1.2, -0.8, -0.4, 0, 0.4, 0.8)
```

4.1 Point estimates and influence-function-based variance

`f_data_analysis_point()` returns, for each value of α, the estimated mean outcome $\widehat{\mu}(t) = s\{B(t)'\widehat{\beta}\}$ at 3, 6, 9, and 12 months, together with the influence-function-based variance estimate.

```
result_trt_log_weight1 <- run_or_load("result_trt_log_weight1",
  f_data_analysis_point(data      = data_ARC_trt_continuous,
                        link_type = "log",       # "log", "logit", "identity"
                        Alpha_seq = Alpha_seq,   # values of alpha
                        IF_type   = "weight1",   # "weight1" or "weight2"
                        SIM_model = "norm1coef")
)
```

The output is a list of two matrices. Each row corresponds to one value of α, and the columns correspond to the target times.

```
# point estimates of mu(t) at 3, 6, 9 and 12 months
result_trt_log_weight1[[1]]
```

```
##      Alpha   month3   month6   month9  month12
## [1,]  -1.2 1.266921 1.302730 1.193441 1.145256
## [2,]  -0.8 1.506160 1.499981 1.422928 1.369032
## [3,]  -0.4 1.822623 1.748302 1.722639 1.675810
## [4,]   0.0 2.238196 2.076359 2.107548 2.083038
## [5,]   0.4 2.737782 2.518863 2.582039 2.572956
## [6,]   0.8 3.270377 3.078076 3.128075 3.095330
```

```
# influence-function-based variance estimates
result_trt_log_weight1[[2]]
```

```
##      Alpha      month3      month6      month9     month12
## [1,]  -1.2 0.007003940 0.002694925 0.005702619 0.009368380
```

```
## [2,]  -0.8 0.006828712 0.003061730 0.005488218 0.008517975
## [3,]  -0.4 0.008471687 0.004208197 0.006554048 0.009993466
## [4,]   0.0 0.013338219 0.006710440 0.009656670 0.014092929
## [5,]   0.4 0.019020171 0.010424194 0.015245384 0.019375519
## [6,]   0.8 0.020559845 0.013792384 0.024364191 0.029366297
```

These can be used to construct Wald confidence intervals with the influence-function-based variance:

```
est <- result_trt_log_weight1[[1]][, -1]
se  <- sqrt(result_trt_log_weight1[[2]][, -1])

IF_CI <- data.frame(Alpha    = rep(Alpha_seq, times = 4),
                    Month    = rep(c(3, 6, 9, 12), each = length(Alpha_seq)),
                    Estimate = as.vector(est),
                    Lower_ci = as.vector(est - qnorm(0.975) * se),
                    Upper_ci = as.vector(est + qnorm(0.975) * se))
head(IF_CI)
```

```
##   Alpha Month Estimate Lower_ci Upper_ci
## 1  -1.2     3 1.266921 1.102893 1.430950
## 2  -0.8     3 1.506160 1.344196 1.668123
## 3  -0.4     3 1.822623 1.642224 2.003021
## 4   0.0     3 2.238196 2.011837 2.464554
## 5   0.4     3 2.737782 2.467477 3.008088
## 6   0.8     3 3.270377 2.989344 3.551411
```

4.2 Parametric bootstrap confidence intervals

As an alternative to the influence-function-based variance, we can use the parametric bootstrap. We first generate $B = 200$ parametric bootstrap data sets with `inner_ARC_SIR_sim_data_count_paraboot()`, and then estimate $\mu(t)$ in each of them with `f_data_analysis_line()`. Here, setting `spline_seq_orig = c(90, 180, 270, 360)` returns the estimates at 3, 6, 9, and 12 months.

```
set.seed(123)  # for the treatment group
B <- 200       # number of parametric bootstrap data sets
Index_sim <- sample(1:50000000, B, replace = FALSE)

result_simu <- run_or_load("paraboot_data_trt",
  parallel::mclapply(Index_sim,
                     inner_ARC_SIR_sim_data_count_paraboot,
                     SIM_model = "norm1coef",
                     data      = data_ARC_trt_continuous,
                     mc.cores  = n_cores)
)

# apply an estimation function to the index-th bootstrap data set
call_function_with_data <- function(index, fun, ...) {
  data <- result_simu[[index]][[1]]
  fun(data, ...)
```

```
}

result_trt <- run_or_load("paraboot_est_trt",
  parallel::mclapply(seq_len(B),
                     call_function_with_data,
                     f_data_analysis_line,
                     link_type       = "log",
                     IF_type         = "weight1",
                     SIM_model       = "norm1coef",
                     Alpha_seq       = Alpha_seq,
                     spline_seq_orig = c(90, 180, 270, 360),
                     mc.cores        = n_cores)
)
```

With `link = "log"`, `paraboot_var()` computes the parametric bootstrap variance of $\log \widehat{\mu}(t)$ for a given α and month. Given the point estimate and this variance on the log scale, `Wald_CI()` returns the point estimate and the 95% Wald confidence interval on the log scale, and `exp()` transforms them back to the original scale. For example, for month 3 and $\alpha = -1.2$:

```
SIM_treat_03 <- result_trt_log_weight1[[1]][, 2]   # estimates at month 3
month <- 3
alpha <- -1.2

refer_mean   <- SIM_treat_03 |> log()
var_paraboot <- paraboot_var(data = result_trt, alpha, month,
                             Alpha_seq = Alpha_seq, link = "log")
mean_orig    <- refer_mean[which(Alpha_seq == alpha)]

t_SIM_treat_03 <- Wald_CI(mean_orig, var_orig = var_paraboot) |> exp()
t_SIM_treat_03   # point estimate, lower bound, upper bound
```

```
## [1] 1.266921 1.066780 1.504610
```

5 Visualization

5.1 Estimated mean curves

To draw the mean curves, we estimate $\mu(t)$ on the full time grid $t \in \{60, 61, \ldots, 460\}$, which is the default of `f_data_analysis_line()`. This function returns point estimates only. The dotted lines show the plausible range $(1.2, 3.0)$ for the mean ACQ score given by the clinical collaborators.

```
spline_seq  <- seq(60, 460, by = 1)
range_value <- c(1.2, 3)

df_trt <- run_or_load("line_est_trt",
  f_data_analysis_line(data      = data_ARC_trt_continuous,
                       link_type = "log",
                       Alpha_seq = Alpha_seq,
```

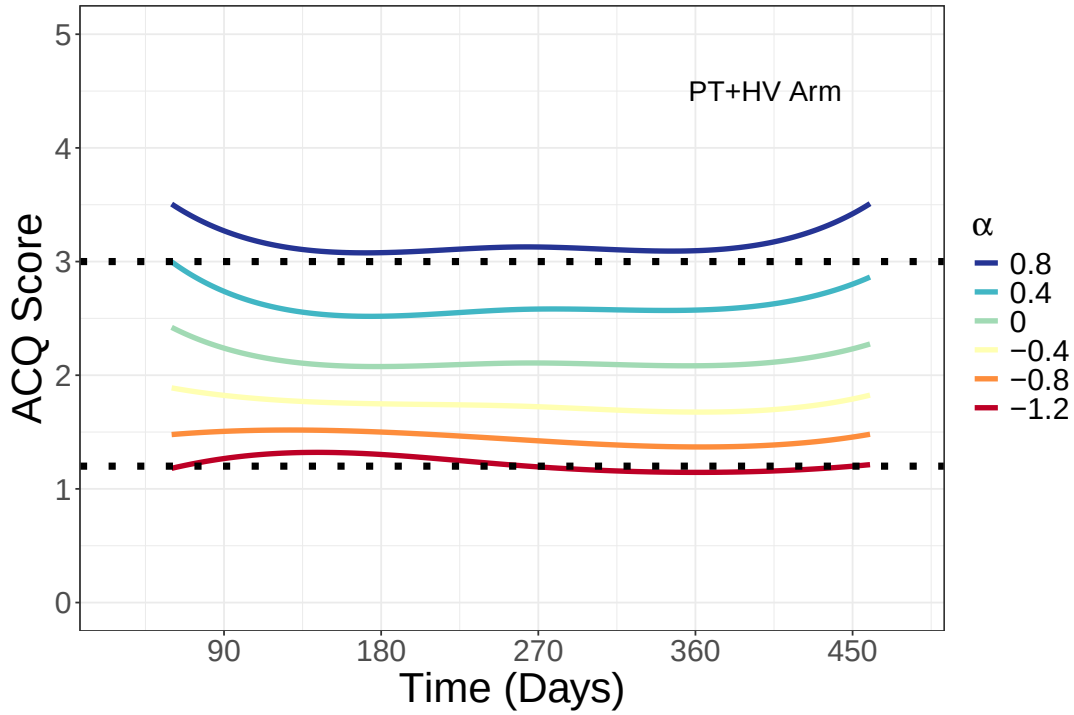
```
                        IF_type   = "weight1",
                        SIM_model = "norm1coef")
)
```

```
plot_df_trt <- data.frame(
  Time  = rep(spline_seq, length(Alpha_seq)),
  Mean  = as.vector(t(df_trt[[1]][, -1])),
  Alpha = rep(as.character(Alpha_seq), each = length(spline_seq))
)

alpha_colours <- c("#253494", "#41b6c4", "#a1dab4",
                   "#ffffb2", "#fd8d3c", "#bd0026")
alpha_breaks  <- c("0.8", "0.4", "0", "-0.4", "-0.8", "-1.2")

line_plot_trt <- ggplot(plot_df_trt,
                        aes(x = Time, y = Mean, colour = Alpha)) +
  geom_line(linewidth = 1.5) +
  theme_bw() +
  expand_limits(x = c(30, 480), y = c(0, 5)) +
  scale_x_continuous(breaks = seq(90, 450, by = 90)) +
  scale_colour_manual(values = alpha_colours, breaks = alpha_breaks) +
  labs(x = "Time (Days)", y = "ACQ Score", colour = expression(alpha)) +
  theme(axis.title   = element_text(size = 25),
        legend.title = element_text(size = 20),
        axis.text    = element_text(size = 18),
        legend.text  = element_text(size = 18)) +
  annotate("text", x = 400, y = 4.5, label = "PT+HV Arm", size = 6) +
  geom_hline(yintercept = range_value, linetype = "dotted", linewidth = 2)

line_plot_trt
```



5.2 Confidence intervals at the target times

Next, we compute the parametric bootstrap Wald confidence intervals for all values of α at 3, 6, 9, and 12 months. The helper below repeats the calculation above for each α at a given month. The points are slightly shifted horizontally so that the intervals do not overlap.

```
Orig_res_trt <- result_trt_log_weight1
months       <- c(3, 6, 9, 12)
centre_days  <- c(90, 180, 270, 360)

ci_by_month <- function(month, paraboot_res, orig_res) {
  refer_mean <- orig_res[[1]][, which(months == month) + 1] |> log()
  out <- t(sapply(Alpha_seq, function(alpha) {
    mean_orig    <- refer_mean[which(Alpha_seq == alpha)]
    var_paraboot <- paraboot_var(data = paraboot_res, alpha, month,
                                 Alpha_seq = Alpha_seq, link = "log")
    Wald_CI(mean_orig, var_orig = var_paraboot) |> exp()
  }))
  data.frame(Alpha    = as.character(Alpha_seq),
             Time     = centre_days[months == month] + seq(-15, 10, by = 5),
             Estimate = out[, 1],
             Lower_ci = out[, 2],
             Upper_ci = out[, 3])
}

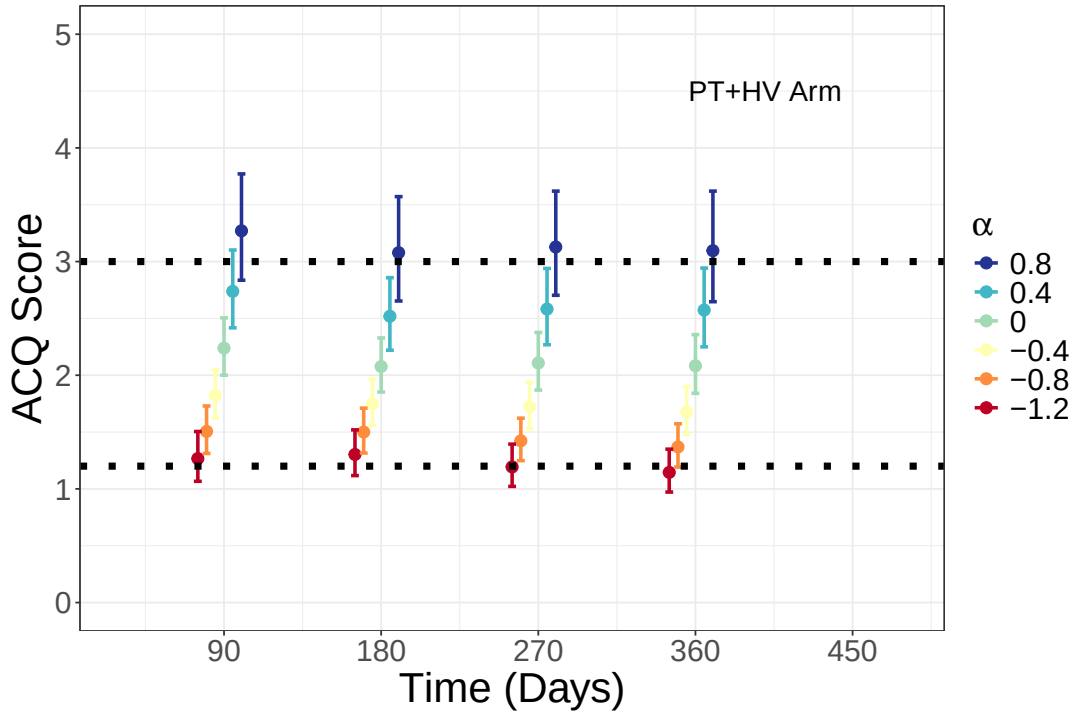
plot_df_SIM_treat <- do.call(rbind, lapply(months, ci_by_month,
                                           paraboot_res = result_trt,
```

```
                                          orig_res     = Orig_res_trt))
plot_df_SIM_treat
```

```
##    Alpha Time Estimate  Lower_ci Upper_ci
## 1   -1.2   75 1.266921 1.0667805 1.504610
## 2   -0.8   80 1.506160 1.3116243 1.729548
## 3   -0.4   85 1.822623 1.6223680 2.047596
## 4      0   90 2.238196 2.0001090 2.504623
## 5    0.4   95 2.737782 2.4167532 3.101455
## 6    0.8  100 3.270377 2.8360363 3.771238
## 7   -1.2  165 1.302730 1.1170128 1.519325
## 8   -0.8  170 1.499981 1.3155042 1.710328
## 9   -0.4  175 1.748302 1.5556868 1.964765
## 10     0  180 2.076359 1.8518002 2.328148
## 11   0.4  185 2.518863 2.2199189 2.858065
## 12   0.8  190 3.078076 2.6530415 3.571204
## 13  -1.2  255 1.193441 1.0218179 1.393891
## 14  -0.8  260 1.422928 1.2483607 1.621907
## 15  -0.4  265 1.722639 1.5295626 1.940087
## 16     0  270 2.107548 1.8693106 2.376148
## 17   0.4  275 2.582039 2.2680182 2.939539
## 18   0.8  280 3.128075 2.7032553 3.619656
## 19  -1.2  345 1.145256 0.9720458 1.349330
## 20  -0.8  350 1.369032 1.1918779 1.572517
## 21  -0.4  355 1.675810 1.4777438 1.900425
## 22     0  360 2.083038 1.8408676 2.357067
## 23   0.4  365 2.572956 2.2496143 2.942773
## 24   0.8  370 3.095330 2.6468548 3.619793
```

```
CI_plot_trt <- ggplot(plot_df_SIM_treat,
                      aes(x = Time, y = Estimate, colour = Alpha)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = Lower_ci, ymax = Upper_ci), linewidth = 1) +
  theme_bw() +
  expand_limits(x = c(30, 480), y = c(0, 5)) +
  scale_x_continuous(breaks = seq(90, 450, by = 90)) +
  scale_colour_manual(values = alpha_colours, breaks = alpha_breaks) +
  labs(x = "Time (Days)", y = "ACQ Score", colour = expression(alpha)) +
  theme(axis.title   = element_text(size = 25),
        legend.title = element_text(size = 20),
        axis.text    = element_text(size = 18),
        legend.text  = element_text(size = 18)) +
  annotate("text", x = 400, y = 4.5, label = "PT+HV Arm", size = 6) +
  geom_hline(yintercept = range_value, linetype = "dotted", linewidth = 2)

CI_plot_trt
```



The same code applies to the control arm after replacing `data_ARC_trt_continuous` with `data_ARC_crl_continuous`.

5.3 Contour plots of the treatment effect

To study the treatment effect $\delta(t) = \mu^1(t) - \mu^0(t)$ under different combinations of (α_0, α_1), we estimate $\mu^a(t)$ in **both** arms on a fine grid $\alpha \in \{-1.2, -1.18, \ldots, 0.8\}$, using both the original data and $B = 200$ parametric bootstrap data sets.

5.3.1 Estimation on a fine grid of α

```
Alpha_seq_contour <- seq(-1.2, 0.8, by = 0.02)
```

```
# treatment group: original data
result_trt_log_weight1_contour <- run_or_load("contour_orig_trt",
  f_data_analysis_line(data            = data_ARC_trt_continuous,
                       Alpha_seq       = Alpha_seq_contour,
                       spline_seq_orig = c(90, 180, 270, 360),
                       link_type       = "log",
                       IF_type         = "weight1",
                       SIM_model       = "norm1coef")
)

# treatment group: parametric bootstrap
set.seed(456)
B <- 200
Index_sim <- sample(1:50000000, B, replace = FALSE)
```

```
result_simu <- run_or_load("contour_paraboot_data_trt",
  parallel::mclapply(Index_sim,
                     inner_ARC_SIR_sim_data_count_paraboot,
                     data      = data_ARC_trt_continuous,
                     SIM_model = "norm1coef",
                     mc.cores  = n_cores)
)

result_trt_contour <- run_or_load("contour_paraboot_est_trt",
  parallel::mclapply(seq_len(B),
                     call_function_with_data,
                     f_data_analysis_line,
                     link_type       = "log",
                     IF_type         = "weight1",
                     SIM_model       = "norm1coef",
                     spline_seq_orig = c(90, 180, 270, 360),
                     Alpha_seq       = Alpha_seq_contour,
                     mc.cores        = n_cores)
)
```

```
# control group: original data
result_crl_log_weight1_contour <- run_or_load("contour_orig_crl",
  f_data_analysis_line(data            = data_ARC_crl_continuous,
                       Alpha_seq       = Alpha_seq_contour,
                       spline_seq_orig = c(90, 180, 270, 360),
                       link_type       = "log",
                       IF_type         = "weight1",
                       SIM_model       = "norm1coef")
)

# control group: parametric bootstrap
set.seed(345)
B <- 200
Index_sim <- sample(1:50000000, B, replace = FALSE)

result_simu <- run_or_load("contour_paraboot_data_crl",
  parallel::mclapply(Index_sim,
                     inner_ARC_SIR_sim_data_count_paraboot,
                     data      = data_ARC_crl_continuous,
                     SIM_model = "norm1coef",
                     mc.cores  = n_cores)
)

result_crl_contour <- run_or_load("contour_paraboot_est_crl",
  parallel::mclapply(seq_len(B),
                     call_function_with_data,
```

```
                    f_data_analysis_line,
                    link_type       = "log",
                    IF_type         = "weight1",
                    SIM_model       = "norm1coef",
                    spline_seq_orig = c(90, 180, 270, 360),
                    Alpha_seq       = Alpha_seq_contour,
                    mc.cores        = n_cores)
)
```

5.3.2 Treatment effect estimates and confidence intervals

For each pair (α_1, α_0), `Wald_CI_effect()` returns the estimated treatment effect $\widehat{\delta}(t)$ and its 95% Wald confidence interval, where the variance is estimated from the parametric bootstrap estimates of $\widehat{\mu}^1(t) - \widehat{\mu}^0(t)$.

Note that `Wald_CI_effect()` reads the objects `data_trt`, `data_crl`, `SIM_treat`, and `SIM_control` from the global environment, so they must be defined with exactly these names before it is called.

```
# parametric bootstrap estimates
data_trt    <- result_trt_contour
data_crl    <- result_crl_contour
# estimates from the original data
SIM_treat   <- result_trt_log_weight1_contour[[1]]
SIM_control <- result_crl_log_weight1_contour[[1]]

contour_df <- function(month) {
  grid <- expand.grid(Alpha_crl = Alpha_seq_contour,
                      Alpha_trt = Alpha_seq_contour)
  res  <- t(mapply(function(a_trt, a_crl) {
    Wald_CI_effect(alpha_trt = a_trt,
                   alpha_crl = a_crl,
                   month     = month,
                   Alpha_seq = Alpha_seq_contour)
  }, grid$Alpha_trt, grid$Alpha_crl))
  # the last two columns are the lower and upper confidence limits
  k <- ncol(res)
  data.frame(Alpha_trt    = grid$Alpha_trt,
             Alpha_crl    = grid$Alpha_crl,
             Estimate_vec = res[, 1],
             Lower_ci     = res[, k - 1],
             Upper_ci     = res[, k])
}

df_m6  <- contour_df(month = 6)
df_m12 <- contour_df(month = 12)
head(df_m6)
```

```
##   Alpha_trt Alpha_crl Estimate_vec   Lower_ci  Upper_ci
```

```
## 1      -1.2     -1.20    0.3924939 0.14322609 0.6417617
## 2      -1.2     -1.18    0.3813163 0.13196721 0.6306653
## 3      -1.2     -1.16    0.3699326 0.12052900 0.6193362
## 4      -1.2     -1.14    0.3583388 0.10865434 0.6080232
## 5      -1.2     -1.12    0.3465306 0.09628647 0.5967747
## 6      -1.2     -1.10    0.3345038 0.08415377 0.5848538
```

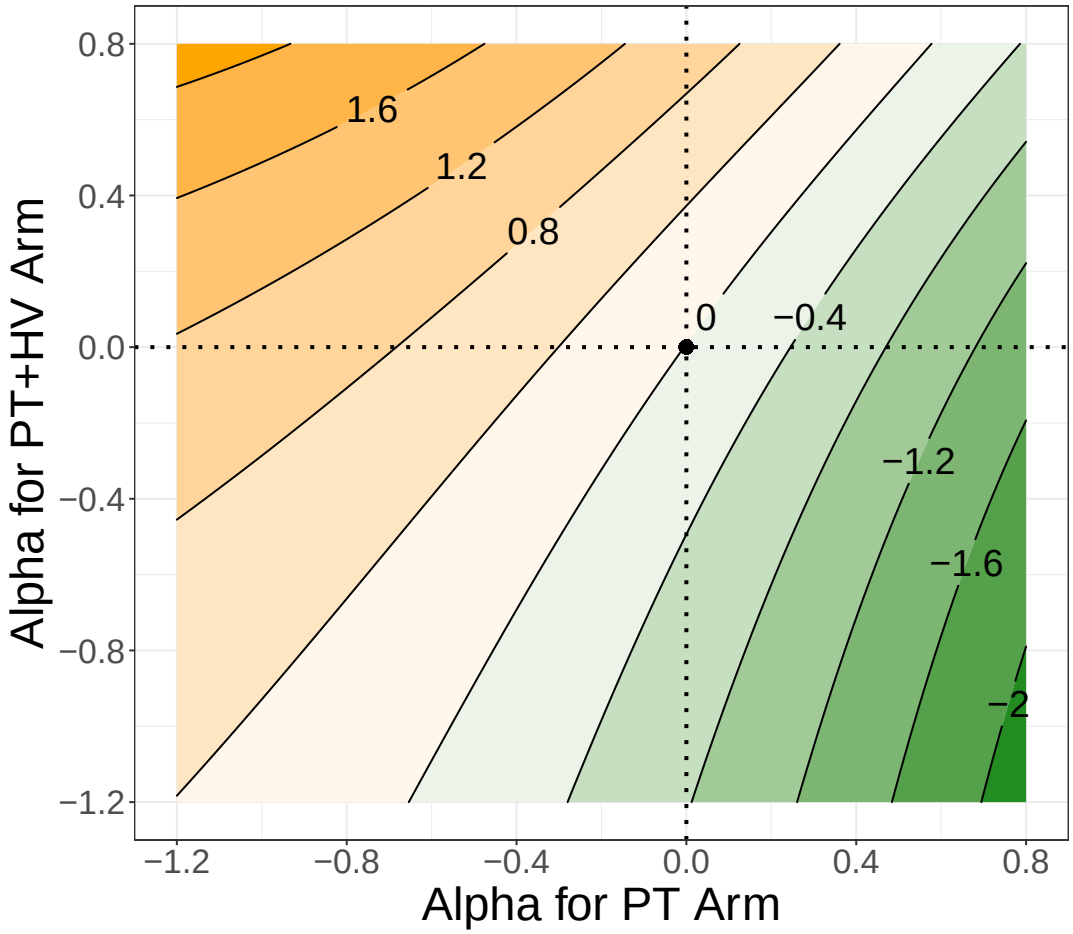
```r
# sanity check: lower limit <= estimate <= upper limit
ci_ok <- function(df) {
  with(df, all(Lower_ci <= Estimate_vec & Estimate_vec <= Upper_ci))
}
stopifnot(ci_ok(df_m6), ci_ok(df_m12))
```

5.3.3 Contour plot of the point estimates

The first plot shows the estimated treatment effect at 6 months for each combination of α_0 (PT arm, x-axis) and α_1 (PT+HV arm, y-axis). The dot at $(0, 0)$ corresponds to the explainable assessment assumption in both arms.

```r
SIM_p1 <- ggplot(df_m6,
                 aes(x = Alpha_crl, y = Alpha_trt, z = Estimate_vec)) +
  geom_contour_fill(binwidth = 0.4, show.legend = FALSE) +
  scale_fill_divergent(mid = "white", low = "forestgreen",
                       high = "orange") +
  geom_contour2(aes(z = Estimate_vec, label = after_stat(level)),
                colour = "black", size = 0.5, binwidth = 0.4, skip = 0.4,
                margin = grid::unit(c(5, 5, 5, 5), "pt"),
                label.placer = label_placer_fraction(
                  frac = 0.38, rot_adjuster = angle_fixed()),
                label_size = 7) +
  theme_bw() +
  labs(x = "Alpha for PT Arm", y = "Alpha for PT+HV Arm") +
  theme(axis.title = element_text(size = 25),
        axis.text  = element_text(size = 18)) +
  geom_vline(xintercept = 0, linetype = "dotted", linewidth = 1) +
  geom_hline(yintercept = 0, linetype = "dotted", linewidth = 1) +
  geom_point(x = 0, y = 0, size = 3) +
  scale_x_continuous(breaks = Alpha_seq) +
  scale_y_continuous(breaks = Alpha_seq)

SIM_p1
```



5.3.4 Contour plot of the confidence intervals

The next two plots summarize statistical significance at 6 and 12 months, respectively. For each (α_0, α_1), we record the confidence limit closest to 0 when the interval excludes 0, and 0 otherwise:

- `CI_info` = lower bound, if the lower bound > 0 (the PT+HV arm has a higher mean ACQ score);
- `CI_info` = upper bound, if the upper bound < 0 (the PT+HV arm has a lower mean ACQ score, i.e., better asthma control);
- `CI_info` = 0, if the interval contains 0.

White regions therefore indicate insufficient evidence of a treatment effect. Because a lower ACQ score indicates better asthma control, green regions favor the home visits intervention.

```
# confidence limit closest to 0 if the CI excludes 0, and 0 otherwise
ci_info <- function(df) {
  with(df, Lower_ci * (Lower_ci > 0) + Upper_ci * (Upper_ci < 0))
}

plot_ci_contour <- function(df, title) {
  df2 <- mutate(df, CI_info = ci_info(df))
  ggplot(df2, aes(x = Alpha_crl, y = Alpha_trt, z = CI_info)) +
```

```
    geom_contour_fill(breaks = seq(-2.6, 2.6, by = 0.4)) +
    scale_fill_divergent(mid = "white", low = "forestgreen",
                         high = "orange", midpoint = 0) +
    theme_bw() +
    theme(legend.title = element_blank()) +
    labs(x = "Alpha for PT Arm", y = "Alpha for PT+HV Arm", title = title) +
    theme(plot.title      = element_text(size = 25),
          axis.title      = element_text(size = 25),
          axis.text       = element_text(size = 18),
          legend.text     = element_text(size = 14),
          legend.key.size = unit(1, "cm")) +
    geom_vline(xintercept = 0, linetype = "dotted", linewidth = 1) +
    geom_hline(yintercept = 0, linetype = "dotted", linewidth = 1) +
    geom_point(x = 0, y = 0, size = 3) +
    scale_x_continuous(breaks = Alpha_seq) +
    scale_y_continuous(breaks = Alpha_seq)
}

# check that the breaks cover the range of CI_info
range(ci_info(df_m6))
```

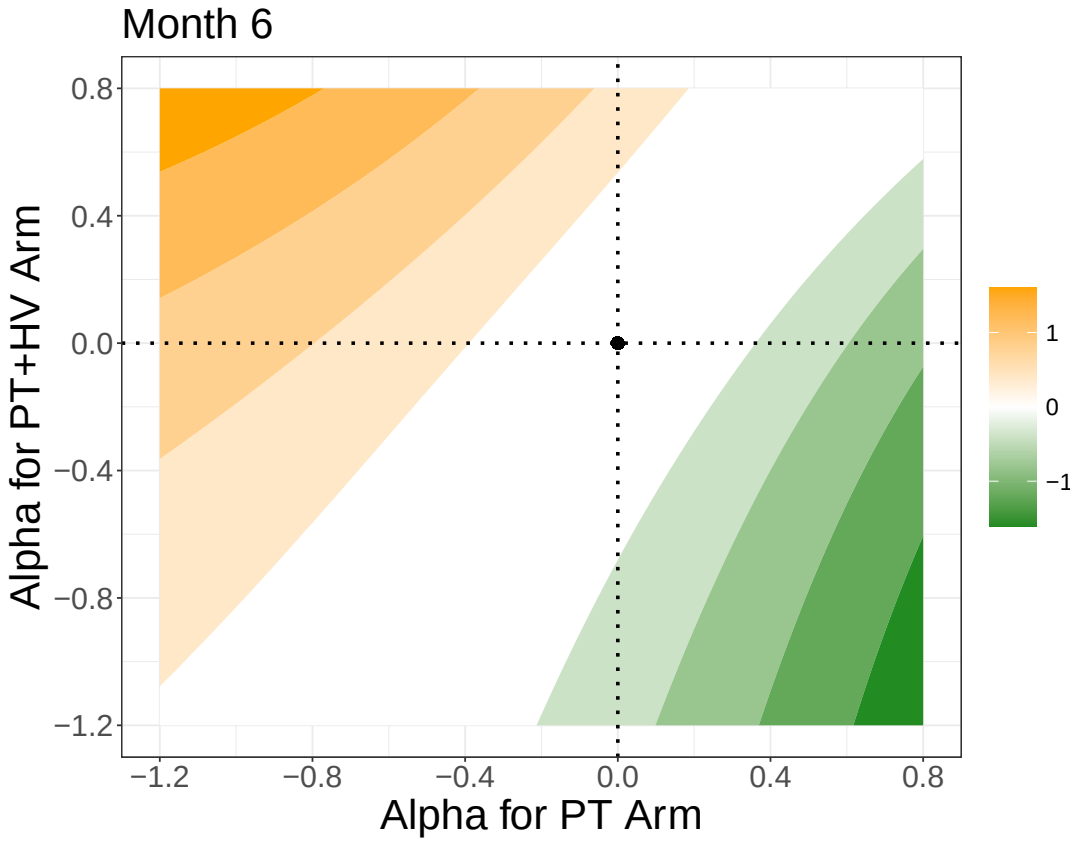
```
## [1] -1.705141  1.691407
```

```
range(ci_info(df_m12))
```

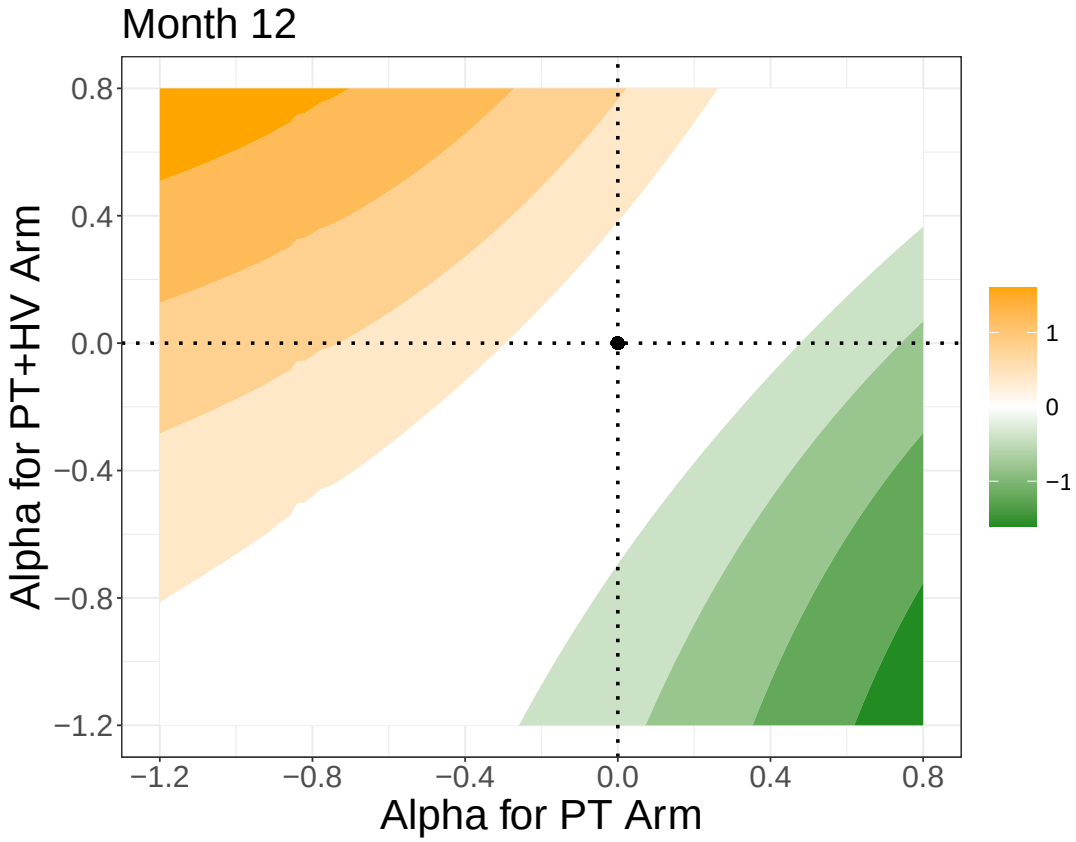
```
## [1] -1.661636  1.688590
```

```
SIM_p2_m6  <- plot_ci_contour(df_m6,  title = "Month 6")
SIM_p2_m12 <- plot_ci_contour(df_m12, title = "Month 12")

SIM_p2_m6
```


Month 6
Alpha for PT+HV Arm
0.8
0.4
0.0
−0.4
−0.8
−1.2
−1.2
−0.8
−0.4
0.0
0.4
0.8
Alpha for PT Arm
1
0
−1

```
SIM_p2_m12
```



If the range of `CI_info` printed above falls outside $[-2.6, 2.6]$, widen the `breaks` in `geom_contour_fill()`.

6 Selection of the sensitivity parameter

We now estimate the range (α_a^l, α_a^u) of sensitivity parameter values under which the mean ACQ score stays within the plausible range $(\mu_a^l, \mu_a^u) = (1.2, 3.0)$ for all $t \in [60, 460]$. We use the initial search grid $\mathcal{A}_M = \{-3, -2.99, \ldots, 3\}$. The time grid $\mathcal{T}_K$ is handled internally by Gauss–Legendre quadrature nodes on $[60, 460]$. We illustrate the procedure for the treatment arm with the log link and weight function $W_1(t; \beta)$. The control arm is handled in the same way.

6.1 Point estimates and bias-corrected point estimates

6.1.1 Algorithm 1: original point estimates

`fun_est_IF_onestep_analysis()` implements Algorithm 1. For each $\alpha \in \mathcal{A}_M$, it solves the estimating equation for $\widehat{\beta}^a(\alpha)$ and stores the influence-function-based information needed by the bootstrap procedures (the estimated influence functions and the inverse of their derivatives). The first element of the output contains the original point estimates $(\widehat{\alpha}_a^l, \widehat{\alpha}_a^u)$. If no value in $\mathcal{A}_M$ satisfies the constraint, the output is `NA`, and Algorithms 2 and 3 cannot be performed.

```
orig_trt_log_weight1 <- run_or_load("alg1_orig_trt",
  fun_est_IF_onestep_analysis(df_sim    = data_ARC_trt_continuous,
                              Alpha_seq = seq(-3, 3, by = 0.01),
                              range     = c(1.2, 3),
```

```
                                link_type = "log",
                                SIM_model = "norm1coef",
                                IF_type   = "weight1")
)
```

6.1.2 Algorithm 2: multiplier bootstrap bias correction

`f_onestep_bias_adaptive()` implements one replicate of Algorithm 2 with the one-step multiplier bootstrap estimator. The main options are:

- `weight_type`: the distribution of the multiplier weights, `"rademacher"`, `"normal"`, or `"exp"`;
- `search_method`: `"grid"` evaluates all values in $\mathcal{A}_M$; `"binary"` uses binary search based on monotonicity; `"adaptive"` (recommended) uses binary search first and switches to grid search if NA values occur.

```
search_method <- "adaptive"

set.seed(123)
Boot_num <- 200
Boot_seed_all <- sample(500000000, size = Boot_num, replace = TRUE)

result_bias <- run_or_load("alg2_bias_trt",
  parallel::mclapply(Boot_seed_all,
                     f_onestep_bias_adaptive,
                     orig_input    = orig_trt_log_weight1,
                     Alpha_seq     = seq(-3, 3, by = 0.01),
                     weight_type   = "rademacher",
                     link_type     = "log",
                     search_method = search_method,
                     range         = c(1.2, 3),
                     mc.cores      = n_cores)
)
```

Each replicate returns the bootstrap estimates $(\widehat{\alpha}_a^{l,b}, \widehat{\alpha}_a^{u,b})$ in its first element and the search method that was actually used in its second element. The bias-corrected estimates are

$$\widehat{\alpha}_a^{l,corr} = 2\widehat{\alpha}_a^{l} - \frac{1}{B}\sum_{b=1}^{B}\widehat{\alpha}_a^{l,b}, \qquad \widehat{\alpha}_a^{u,corr} = 2\widehat{\alpha}_a^{u} - \frac{1}{B}\sum_{b=1}^{B}\widehat{\alpha}_a^{u,b}.$$

Replicates that fail to identify a feasible α are recorded as `NA` and excluded.

```
Alpha_l_trt_point_orig <- orig_trt_log_weight1[[1]][1]
Alpha_u_trt_point_orig <- orig_trt_log_weight1[[1]][2]

corr_trt_log_weight1 <- result_bias

Boot_est_l <- rep(NA, Boot_num)
Boot_est_u <- rep(NA, Boot_num)
Type       <- rep(NA, Boot_num)
```

```
for (j in 1:Boot_num) {
  temp <- corr_trt_log_weight1[[j]]
  Boot_est_l[j] <- temp[[1]][1]
  Boot_est_u[j] <- temp[[1]][2]
  Type[j]       <- temp[[2]]
}

Alpha_l_trt_point_corr <- 2 * Alpha_l_trt_point_orig -
  mean(Boot_est_l, na.rm = TRUE)
Alpha_u_trt_point_corr <- 2 * Alpha_u_trt_point_orig -
  mean(Boot_est_u, na.rm = TRUE)

# original point estimates of (alpha_l, alpha_u)
trt_log_weight1_orig <- c(Alpha_l_trt_point_orig, Alpha_u_trt_point_orig)
trt_log_weight1_orig
```

```
## [1] -1.09  0.40
```

```
# bias-corrected point estimates of (alpha_l, alpha_u)
trt_log_weight1_corr <- c(Alpha_l_trt_point_corr, Alpha_u_trt_point_corr)
trt_log_weight1_corr
```

```
## [1] -1.20610  0.46655
```

```
# search method used in each bootstrap replicate
table(Type)
```

```
## Type
## binary
##    200
```

6.2 Confidence set for (α_a^l, α_a^u)

6.2.1 Algorithm 3: double bootstrap

`f_double_boot_onestep_adaptive()` implements one outer replicate of Algorithm 3, including `L` inner replicates. We use $B_1 = 400$ outer and $B_2 = 50$ inner replicates. This is the most time-consuming step. Because the result is loaded from the cache here, the reported running time is close to zero; it reflects the actual computation time only on the first run, when no saved result exists.

```
orig_input <- orig_trt_log_weight1

Boot_outer    <- 400
Boot_inner    <- 50
search_method <- "adaptive"

set.seed(567)
Boot_seed_all <- sample(500000000, size = Boot_outer, replace = TRUE)
```

```
# the running time is meaningful only on the first run (no saved result)
start <- Sys.time()
result_double <- run_or_load("alg3_double_trt",
  parallel::mclapply(Boot_seed_all,
                     f_double_boot_onestep_adaptive,
                     orig_input    = orig_input,
                     Alpha_seq     = seq(-3, 3, by = 0.01),
                     range         = c(1.2, 3),
                     weight_type   = "rademacher",
                     link_type     = "log",
                     IF_type       = "weight1",
                     search_method = search_method,
                     L             = Boot_inner,   # inner bootstrap size
                     mc.cores      = n_cores)
)
end <- Sys.time()
end - start
```

```
## Time difference of 0.001513004 secs
```

Each outer replicate b_1 returns a list with five elements:

1. the outer bootstrap estimates $(\widehat{\alpha}_a^{l,b_1}, \widehat{\alpha}_a^{u,b_1})$;
2. the inner bootstrap estimates $\{\widehat{\alpha}_a^{l,b_1,b_2}\}_{b_2=1}^{B_2}$;
3. the inner bootstrap estimates $\{\widehat{\alpha}_a^{u,b_1,b_2}\}_{b_2=1}^{B_2}$;
4. the number of inner replicates with `NA`, for the lower and the upper bound;
5. the search method that was actually used.

We then follow Steps 2–4 of Algorithm 3:

- Bias-corrected outer estimates: $\widehat{\alpha}_a^{l,b_1,corr} = 2\widehat{\alpha}_a^{l,b_1} - B_2^{-1}\sum_{b_2}\widehat{\alpha}_a^{l,b_1,b_2}$, and similarly for the upper bound.
- Bias-corrected point estimates: $\widehat{\alpha}_a^{l,corr} = 2\widehat{\alpha}_a^{l} - B_1^{-1}\sum_{b_1}\widehat{\alpha}_a^{l,b_1}$, and similarly for the upper bound.
- 95% confidence sets:
  - **Basic bootstrap**: $(2\widehat{\alpha}_a^{l,corr} - C_{0.975}^{l},\ 2\widehat{\alpha}_a^{u,corr} - C_{0.025}^{u})$, where $C_{0.975}^{l}$ is the 97.5% quantile of $\{\widehat{\alpha}_a^{l,b_1,corr}\}$ and $C_{0.025}^{u}$ is the 2.5% quantile of $\{\widehat{\alpha}_a^{u,b_1,corr}\}$.
  - **Wald**: $(\widehat{\alpha}_a^{l,corr} - q_{0.975}\,\mathrm{sd}\{\widehat{\alpha}_a^{l,b_1,corr}\},\ \widehat{\alpha}_a^{u,corr} + q_{0.975}\,\mathrm{sd}\{\widehat{\alpha}_a^{u,b_1,corr}\})$, where $q_{0.975}$ is the 97.5% quantile of the standard normal distribution.

For comparison, we also compute the same confidence sets without bias correction.

```
Orig <- orig_trt_log_weight1
temp <- result_double

alpha_u_boot_outer_orig <- rep(NA, Boot_outer)
alpha_l_boot_outer_orig <- rep(NA, Boot_outer)
alpha_u_boot_outer_corr <- rep(NA, Boot_outer)
alpha_l_boot_outer_corr <- rep(NA, Boot_outer)
```

```
NA_count_l_i <- rep(NA, Boot_outer)
NA_count_u_i <- rep(NA, Boot_outer)
Type         <- rep(NA, Boot_outer)

for (j in 1:Boot_outer) {
  alpha_l_boot_outer_orig[j] <- temp[[j]][[1]][1]
  alpha_u_boot_outer_orig[j] <- temp[[j]][[1]][2]
  alpha_l_boot_outer_corr[j] <- 2 * alpha_l_boot_outer_orig[j] -
    mean(temp[[j]][[2]], na.rm = TRUE)
  alpha_u_boot_outer_corr[j] <- 2 * alpha_u_boot_outer_orig[j] -
    mean(temp[[j]][[3]], na.rm = TRUE)

  NA_count_l_i[j] <- temp[[j]][[4]][1]
  NA_count_u_i[j] <- temp[[j]][[4]][2]

  Type[j] <- temp[[j]][[5]]
}

# bias-corrected point estimates
Orig_corr_l <- 2 * Orig[[1]][1] - mean(alpha_l_boot_outer_orig, na.rm = TRUE)
Orig_corr_u <- 2 * Orig[[1]][2] - mean(alpha_u_boot_outer_orig, na.rm = TRUE)

# basic bootstrap confidence set: bias-corrected and original
CI_perc_low_alpha_l_trt_corr1 <- 2 * Orig_corr_l -
  quantile(alpha_l_boot_outer_corr, 0.975, na.rm = TRUE)
CI_perc_upper_alpha_u_trt_corr1 <- 2 * Orig_corr_u -
  quantile(alpha_u_boot_outer_corr, 0.025, na.rm = TRUE)

CI_perc_low_alpha_l_trt_orig1 <- 2 * Orig[[1]][1] -
  quantile(alpha_l_boot_outer_orig, 0.975, na.rm = TRUE)
CI_perc_upper_alpha_u_trt_orig1 <- 2 * Orig[[1]][2] -
  quantile(alpha_u_boot_outer_orig, 0.025, na.rm = TRUE)

# Wald confidence set: bias-corrected and original
CI_perc_low_alpha_l_trt_corr2 <- Orig_corr_l -
  qnorm(0.975) * sd(alpha_l_boot_outer_corr, na.rm = TRUE)
CI_perc_upper_alpha_u_trt_corr2 <- Orig_corr_u +
  qnorm(0.975) * sd(alpha_u_boot_outer_corr, na.rm = TRUE)

CI_perc_low_alpha_l_trt_orig2 <- Orig[[1]][1] -
  qnorm(0.975) * sd(alpha_l_boot_outer_orig, na.rm = TRUE)
CI_perc_upper_alpha_u_trt_orig2 <- Orig[[1]][2] +
  qnorm(0.975) * sd(alpha_u_boot_outer_orig, na.rm = TRUE)
```

The tables below summarize the point estimates and the 95% confidence sets for (α_a^l, α_a^u). The bias-corrected rows correspond to the quantities reported in Table 1 of the paper; note that the paper's results are based on the real ARC data rather than on this simulated data set.

```
selection_summary_point <- data.frame(
  Quantity = c("Original point estimate",
               "Bias-corrected (Algorithm 2; 200 replicates)",
               "Bias-corrected (Algorithm 3; 400 outer replicates)"),
  alpha_l = c(trt_log_weight1_orig[1], trt_log_weight1_corr[1], Orig_corr_l),
  alpha_u = c(trt_log_weight1_orig[2], trt_log_weight1_corr[2], Orig_corr_u),
  row.names = NULL
)
knitr::kable(selection_summary_point, digits = 2, booktabs = TRUE)
```

Quantity	alpha_l	alpha_u
Original point estimate	-1.09	0.40
Bias-corrected (Algorithm 2; 200 replicates)	-1.21	0.47
Bias-corrected (Algorithm 3; 400 outer replicates)	-1.23	0.50

```
selection_summary_set <- data.frame(
  Quantity = c("95% basic bootstrap CI (original)",
               "95% Wald CI (original)",
               "95% basic bootstrap CI (bias-corrected)",
               "95% Wald CI (bias-corrected)"),
  Lower = c(CI_perc_low_alpha_l_trt_orig1,
            CI_perc_low_alpha_l_trt_orig2,
            CI_perc_low_alpha_l_trt_corr1,
            CI_perc_low_alpha_l_trt_corr2),
  Upper = c(CI_perc_upper_alpha_u_trt_orig1,
            CI_perc_upper_alpha_u_trt_orig2,
            CI_perc_upper_alpha_u_trt_corr1,
            CI_perc_upper_alpha_u_trt_corr2),
  row.names = NULL
)
knitr::kable(selection_summary_set, digits = 2, booktabs = TRUE)
```

Quantity	Lower	Upper
95% basic bootstrap CI (original)	-1.44	0.80
95% Wald CI (original)	-1.33	0.74
95% basic bootstrap CI (bias-corrected)	-1.72	1.01
95% Wald CI (bias-corrected)	-1.55	0.93

Finally, we check the number of `NA` values in the inner bootstrap replicates and the search method used in each outer replicate.

```
max(NA_count_l_i)
```

```
## [1] 0
```

```r
max(NA_count_u_i)
```

```
## [1] 0
```

```r
table(Type)
```

```
## Type
## binary
##    400
```

The selected range, together with clinical input, can then be used to choose the values of α_a reported in the main analysis (for example, $\alpha_a \in \{-1.2, -0.8, -0.4, 0, 0.4, 0.8\}$ for the ACQ score).

7 Other outcome types and options

7.1 Binary outcome (AC indicator)

For the binary AC indicator, use the logit link and the corresponding plausible range $(0.2, 0.5)$:

```r
data_ARC_trt_binary <- readRDS(file = "ARC_simu_data_200_logistic_trt")

Alpha_seq_binary <- c(-0.8, -0.4, 0, 0.4, 0.8, 1.2)

result_trt_logit_weight1 <- f_data_analysis_point(
  data      = data_ARC_trt_binary,
  link_type = "logit",
  Alpha_seq = Alpha_seq_binary,
  IF_type   = "weight1",
  SIM_model = "norm1coef"
)

orig_input_binary <- fun_est_IF_onestep_analysis(
  df_sim    = data_ARC_trt_binary,
  Alpha_seq = seq(-3, 3, by = 0.01),
  range     = c(0.2, 0.5),
  SIM_model = "norm1coef",
  link_type = "logit",
  IF_type   = "weight1"
)
```

Three points to note when changing the outcome type:

- When calling `paraboot_var()`, pass `Alpha_seq = Alpha_seq_binary` and `link = "logit"` for the binary outcome.
- For the logit link, use `qlogis()` instead of `log()` to compute `refer_mean`, and `plogis()` instead of `exp()` to transform the point estimate and the confidence interval back to the probability scale.
- Set `range = c(0.2, 0.5)` in all three sensitivity parameter selection functions (`fun_est_IF_onestep_analysis()`, `f_onestep_bias_adaptive()`, and `f_double_boot_onestep_adaptive`

together with `link_type = "logit"`.

7.2 Summary of main options

Argument	Values	Description
`link_type`	`"log"`, `"logit"`, `"identity"`	Link function g in the marginal mean model $\mu(t) = g^{-1}\{B(t)'\beta\}$
`IF_type`	`"weight1"`, `"weight2"`	Weight function $W_1(t;\beta)$ or $W_2(t;\beta)$ in the influence function
`SIM_model`	`"norm1coef"`, `"fixed_coef"`, `"fixed_band"`	Fitting method for the single-index outcome model
`Alpha_seq`	numeric vector	Values of the sensitivity parameter
`spline_seq_orig`	numeric vector	Times (days) at which $\mu(t)$ is reported by `f_data_analysis_line()`
`range`	`c(1.2, 3)`, `c(0.2, 0.5)`, ...	Plausible range (μ_a^l, μ_a^u) of the mean outcome
`weight_type`	`"rademacher"`, `"normal"`, `"exp"`	Distribution of the multiplier weights
`search_method`	`"adaptive"`, `"binary"`, `"grid"`	Search strategy over $\mathcal{A}_M$ in Algorithms 2 and 3
`L`	integer	Number of inner bootstrap replicates in Algorithm 3

8 Session information

```
sessionInfo()
```

```
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sequoia 15.6.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
```

```
## [1] parallel  stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] SensIAT_0.3.0.9000          isoband_0.2.7
##  [3] metR_0.18.0                 ggplot2_3.5.2
##  [5] statmod_1.5.1               BB_2019.10-1
##  [7] orthogonalsplinebasis_0.1.7 dfoptim_2023.1.0
##  [9] MAVE_1.3.11                 survival_3.7-0
## [11] dplyr_1.1.4
##
## loaded via a namespace (and not attached):
##  [1] tidyr_1.3.1        generics_0.1.3     class_7.3-22       KernSmooth_2.23-24
##  [5] lattice_0.22-6     pracma_2.4.4       digest_0.6.37      magrittr_2.0.3
##  [9] evaluate_1.0.3     grid_4.4.2         fastmap_1.2.0      plyr_1.8.9
## [13] Matrix_1.7-1       backports_1.5.0    tinytex_0.55       purrr_1.0.4
## [17] scales_1.3.0       cli_3.6.4          rlang_1.1.5        munsell_0.5.1
## [21] splines_4.4.2      withr_3.0.2        cachem_1.1.0       yaml_2.3.10
## [25] tools_4.4.2        memoise_2.0.1      checkmate_2.3.2    colorspace_2.1-1
## [29] assertthat_0.2.1   vctrs_0.6.5        R6_2.6.1           lifecycle_1.0.4
## [33] MASS_7.3-61        pkgconfig_2.0.3    pillar_1.10.1      gtable_0.3.6
## [37] glue_1.8.0         data.table_1.17.0  Rcpp_1.1.1         xfun_0.51
## [41] tibble_3.2.1       tidyselect_1.2.1   rstudioapi_0.17.1  knitr_1.49
## [45] mda_0.5-5          farver_2.1.2       htmltools_0.5.8.1  labeling_0.4.3
## [49] rmarkdown_2.29     compiler_4.4.2     quadprog_1.5-8
```